Designing E-Commerce Search Ranking: A Complete Derivation Log

> **Source:** An original question from Coupang L6 Staff interview reports. Its actual form in the interview is *"take the project on your résumé and redesign it"* — one candidate's write-up notes they hadn't expected to be asked this.
>
> **Constraints:** P99 end-to-end 200ms, 50K QPS peak, catalog of 500 million products.
>
> **The interviewer is a backend engineer, not an ML practitioner.** Hence the implicit rule throughout: compress the model into a black box and spend the time on the backend problems around it — retrieval, feature serving, index updates, fan-out, tail latency, cost.

---

1. Request Path

```
query
  → NLP (tokenize / spell correction / query features)
  → top aggregator
  → scatter to all index servers
  → retrieval (inverted index)
  → per-shard lightweight ranking, return top N
  → top aggregator merges, re-ranks
  → return a list of doc_ids
```

The last step deserves emphasis: **only doc_ids go back to the frontend.** Full product fields are fetched by the frontend from a separate product service. This keeps the index lean and moves display-layer read pressure out of the ranking system entirely.

---

2. Three Parts of the Index

Structure	Contents	Update characteristics
`term → doc_id list`	Inverted index	Expensive to mutate (rewrites posting lists), but changes very rarely
`doc_id → meta`	Title, commonly used fields	Moderate
`doc_id → feature`	Price, stock, click counts, rating, embeddings	Point updates, cheap, highest change rate

Shard key: `doc_id` hash, 100 shards.

This choice has an important consequence: **all three structures for a given doc live on the same machine.** Features needed by re-ranking therefore require no remote call — read them locally right after retrieval. This is what eliminates the feature-service hop entirely.

The index holds only titles and commonly used fields: roughly 5 TB for 500M products, 50 GB per machine.

---

3. Per-Shard Latency Breakdown

Processing one query on a single index server:

Stage	Work	Time
Retrieval	Intersect posting lists, ~100K candidates	1ms
Coarse rank	Linear scoring (token match), **bucket sort** to top 1,000	2ms
Fine rank	GBDT at 0.5ms each over top 40, select 25	20ms
Total		**22ms**

The bucket-sort step is worth calling out: map float scores onto the integer range `[0, 65535]` and use counting sort for top-K — O(n) rather than O(n log n). 100K candidates resolve in 2ms. Trading a bounded value range for the elimination of comparison sorting is a common technique in retrieval systems and a good detail to volunteer.

Per-machine throughput: 64 cores ÷ 22ms ≈ **3K QPS**.

---

4. Re-Ranking Tier

The top aggregator merges per-shard results, takes the top 500, and runs a heavyweight DNN:

```
2ms each × 500 items       = 1000 core·ms
64 cores in parallel       = 16ms wall clock
Per-machine throughput     = 1000ms / 16ms = 62.5 QPS
```

End-to-end latency: 22 (retrieval) + 16 (re-rank) + ~2ms NLP ≈ **40ms**, leaving 5× headroom against the 200ms P99 budget.

---

5. Embeddings and Features

Computed offline: - Document embeddings — stored alongside the index on the index servers

Computed online: - Query and user embeddings — 1-2ms each, parallel with NLP, adding no latency to the critical path

Real-time signals (price, stock, promotions, click counts, ratings): These do *not* go through a separate feature service. They live in `doc_id → feature` **locally on the index server**. This is the central trade-off of the design:

- **Benefit:** zero remote calls during re-ranking — an entire hop and a component prone to overload are removed
- **Cost:** every change must be pushed to **all replicas** of the owning shard, and a consistency window must be accepted

---

6. Index Updates: Three Pipelines

Pipeline	Contents	Volume	Mechanism
Real-time	Price, stock, promotions, click counters	Largest	KV point updates
Incremental batch	New listings, delistings, title/description edits, review text	Small	Delta segments plus periodic merge
Full rebuild	Schema changes, data repair	Rare	Rebuild offline, then swap

6.1 Real-time pipeline volume

The key point: **every change must be written to all replicas of its shard; replication does not amortise writes.** The divisor is therefore the shard count, not the machine count.

Change-rate assumption	Global QPS	Per shard (100 shards)
1% / second	5,000,000	50,000/s
0.1% / second	**500,000**	**5,000/s**
1% / minute	83,000	833/s

Taking 0.1%/s: **5,000 writes/s per machine**, a light load for an in-memory KV. One caveat worth stating: writes must not degrade read P99 — use batched apply, or separate read and write threads.

> Note in passing: 1%/second implies the entire catalogue turns over every 100 seconds, which is not a credible assumption. **In system design, checking whether an assumption is absurd is often more productive than optimising the design built on it.**

6.2 Engagement signal volume

Estimating with 50M DAU:

Signal	Per user/day	Global QPS
Product views / clicks	30	17,400
Add to cart	2	1,160
Favourites	1	580
Purchases	0.3	174
Reviews	0.02	12
Total		**~19,000/s** (5× peak ≈ 100,000/s)

Across 100 shards: 193/s average per machine, 964/s at peak — an order of magnitude below the price/stock pipeline. These events are naturally spread across documents.

Optimisation: click counts are aggregates and need not be exact in real time. Stream-aggregate over a few-second window and update in batches, collapsing 100 clicks into one write.

6.3 The true mutation rate of the inverted index

Only term additions and removals touch the inverted index:

Event	Frequency	QPS
New product listed	200K/day	2.3
Product delisted	150K/day	1.7
Title/description edit	2M/day	23
New review text	1M/day	12
Total		**~39/s**

Intra-machine fan-out: a single review containing ~100 distinct terms requires updating 100 posting lists. But because sharding is by `doc_id`, those 100 writes **all land on the same machine — they do not cross the network.**

Globally 39 × 100 = 3,900 posting-list writes/s; across 100 shards, roughly 39/s per machine.

That is two orders of magnitude below the 5,000/s of `doc → feature`. This is precisely why the three pipelines must be separated: inverted-index mutations are expensive but rare; feature mutations are cheap but frequent.

> **Hotspot caveat:** when a viral product attracts a burst of reviews, all those writes land on one machine. Tens of reviews per second × 100 terms = a few thousand posting-list writes per second — still survivable on one machine, but worth raising proactively.

---

7. Caching

E-commerce query distributions are extremely head-heavy with a long tail. Assuming an **80%** result-cache hit rate:

```
50K QPS peak → 10K QPS reaching the backend
```

This is the cheapest optimisation in the entire design, cutting machine count to one fifth.

---

8. Machine Count and Cost

Index tier

Every query scatters to all 100 shards, so **each shard must absorb the full backend QPS**:

```
10K QPS ÷ 3K QPS per machine       = 4 replicas
× 2 (peak and utilisation headroom) = 8 replicas
100 shards × 8                      = 800 machines
```

Re-ranking tier

```
10K QPS ÷ 62.5 QPS per machine = 160 machines
× 2                            = 320 machines
```

Roughly 1,120 machines total.

Unit price: 64 cores, 128-256 GB RAM (the index must stay resident), NVMe — approximately $15-20K. Hardware total lands around **$17-22M**; adding racks, power, cooling, networking and operations, three-year TCO is typically 2-3× the hardware price.

> **Common mistakes:** counting machines by shard count alone (omitting replicas), or costing only one tier (omitting re-ranking). Either error shifts the estimate by an order of magnitude.

---

9. Tail Latency: Three Closed-Form Results

A 40ms mean against a 200ms P99 budget looks comfortable, but the scatter-gather structure amplifies the tail. The following three relationships are derivable and require no load testing.

9.1 Fan-out raises the percentile requirement by orders of magnitude

With N independent shards each completing within time t with probability p, the probability all complete is **p^N**.

To hold an overall P99: **p ⩾ 0.99^(1/N)**

Shards N	Each shard must reach
10	P99.90
50	P99.98
100	**P99.99**
1000	P99.999

This is where “fanning out to 100 machines demands P99.99 per shard” comes from — it is derived, not guessed.

9.2 Utilisation drives the tail, with diminishing returns

M/M/1 sojourn time is exponentially distributed, giving **T(x) / T_mean = −ln(1−x)**:

- P95 = 3.0 × mean
- **P99 = 4.6 × mean**
- P99.9 = 6.9 × mean

That ratio is independent of utilisation. But the mean itself equals service time / (1−ρ), which explodes as ρ approaches 1:

Utilisation ρ	P99 / service time
0.3	6.6×
0.5	9.2×
0.7	15×
0.8	23×
0.9	**46×**
0.95	92×

This is the theoretical basis for the "2× headroom" factor — it is not arbitrary. At 90% utilisation the re-ranking tier's 16ms service time implies a 700ms P99, blowing straight through the 200ms budget. At 50% utilisation it is 147ms, just inside.

Diminishing returns: dropping ρ from 0.9 to 0.5 doubles the machine count and improves P99 fivefold; dropping from 0.5 to 0.25 doubles it again for only a 1.5× improvement. In practice one rarely pushes utilisation below 0.5; past that point, other techniques are better value.

9.3 Hedged requests drive the slow probability down exponentially

Issue the same shard request to r replicas and take the first to return: **P(all slow) = (1−p)^r**

Replicas r	P(slow)
1	1%
2	0.01%
3	0.0001%

The two formulas meet exactly: § 9.1 demands P99.99 per shard, and hedging to 2 replicas lifts a P99 to precisely P99.99.

9.4 Practical recommendation

Timeout truncation offers the best value here: set a hard deadline (say 80ms), skip shards that exceed it, and return results from the rest. It caps the tail unconditionally, **depends on no distributional assumption**, and costs one deadline to implement.

The price is that a truncated shard may have held the best product, so a quality metric is required: **how many shards were missing from each query**. How much degradation is acceptable is a product decision.

How to phrase this in an interview: "I wouldn't claim to compute P99 on paper — that needs load testing. But I'd cap the tail with timeout truncation rather than relying on the per-shard latency distribution happening to land inside budget."

---

10. Design Takeaways

1. **The shard key determines the architecture.** Sharding by `doc_id` co-locates term→doc, doc→meta and doc→feature, so re-ranking makes zero remote calls — this removes the entire feature-service hop.
2. **Split update pipelines by cost, not by content.** Inverted-index mutations are expensive but occur tens of times per second; feature mutations are cheap but occur thousands of times per second. Sharing one pipeline lets the expensive path throttle the cheap one.
3. **Replicas do not amortise writes.** The divisor for update volume is the shard count, not the machine count. This is among the

most common errors in capacity estimation.

4. **Check whether the assumption is absurd first.** “1% of products change every second” implies a full catalogue turnover every 100 seconds; correcting the rate makes the problem disappear on its own.
5. **The cost of fan-out is a percentile requirement, not throughput.** N shards each need P99^(1/N); the more shards, the harsher the demand.
6. **Utilisation is the primary lever on tail latency, with diminishing returns.** Past roughly 50% utilisation, switch to hedging and timeouts rather than adding machines.
7. **When explaining an ML system to a backend interviewer, compress the model into a box.** Spend the time on retrieval, features, updates, fan-out and cost around it; expand the model internals only when asked.